

AI활용프로그래밍

Week 13. 여러 일을 동시에

기다리는 동안 다른 일 하기 · 제한 시간 게임 만들기

학습 목표

- 기다리는 동안 다른 일을 시킬 수 있다.
- 쓰레드를 만들고 끝날 때까지 기다릴 수 있다.
- 여럿이 같은 값을 고칠 때 생기는 문제를 안다.
- 제한 시간이 있는 작은 게임을 만들 수 있다.

오늘의 구성

- 1) 기다리는 동안 멈춰 있는 문제
- 2) 쓰레드 만들고 기다리기
- 3) 같은 값을 함께 고치면 생기는 일
- 4) 미니게임 — 로딩 표시와 제한 시간
- 5) With AI — 위험한 곳 찾아내기

기다리는 동안 아무것도 못 합니다

- sleep 은 프로그램 전체를 멈춰 세웁니다.
- 그동안 화면도 안 바뀌고 입력도 못 받습니다.
- 게임이라면 그 시간 동안 얼어붙은 것과 같습니다.
- 그래서 '기다리는 일'을 따로 떼어 내야 합니다.

멈춰 보기

```
import time
print("시작")
time.sleep(1)
print("끝")
```

실행 결과

시작
끝

핵심 포인트

- ▶ 시작이 나오고 1초 동안 아무 일도 없습니다.
- ▶ 그 뒤에야 끝이 나옵니다.
- ▶ 1초를 10초로 바꾸면 답답함이 확 느껴집니다.

동시에 하려면 — 스레드

- 일하는 사람을 한 명 더 두는 것과 같습니다.
- 한 명은 기다리고 다른 한 명은 계속 일합니다.
- 파이썬에서는 이것을 스레드라고 부릅니다.
- 특히 기다림이 많은 일에 효과가 큼니다.



용어 노트

스레드(thread) — 프로그램 안에서 따로 도는 일꾼 하나. 주방에서 한 명이 더 붙는 것과 같습니다.

동시성(concurrency) — 여러 일을 번갈아 가며 진행하는 것. 요리하며 빨래를 돌리는 것과 같습니다.

쓰레드 만들어 실행하기

```
import threading
def work():
    print("작업 중")
t = threading.Thread(target=work)
t.start()
t.join()
print("메인 끝")
```

실행 결과

작업 중

메인 끝

핵심 포인트

- ▶ Thread 에 시킬 일의 이름을 줍니다.
- ▶ 괄호를 붙이지 않습니다. work() 가 아니라 work 입니다.
- ▶ start() 가 일을 시작시킵니다.
- ▶ join() 이 끝날 때까지 기다립니다.

start 와 join 이 하는 일

- `start()` — 새 일꾼을 붙여 일을 시작합니다.
- `join()` — 그 일꾼이 끝날 때까지 기다립니다.
- `join` 을 빼면 메인이 먼저 끝나 버릴 수 있습니다.
- 그래서 출력 순서가 실행할 때마다 달라집니다.

기다리지 않으면 순서가 섞입니다

```
import threading, time

def bg():
    for i in range(3):
        print("tick")
        time.sleep(0.3)

t = threading.Thread(target=bg)
t.start()
print("메인은 계속 갑니다")
t.join()

# 실행할 때마다 순서가
# 달라질 수 있습니다.
```

핵심 포인트

- ▶ 메인이 기다리지 않고 그냥 다음 줄로 갑니다.
- ▶ 그래서 tick 과 메인 문장이 뒤섞입니다.
- ▶ 여러 번 실행해 순서가 바뀌는 것을 보세요.
- ▶ 결과가 매번 달라지니 이 장에는 답을 적지 않았습니다.

코랩에서 스레드를 쓸 때

- 배경에서 찍는 print 가 다음 칸 결과에 섞일 수 있습니다.
- 끝나지 않는 스레드는 칸이 끝나도 계속 돕니다.
- 그래서 끝내는 조건을 반드시 넣어야 합니다.
- 이상하면 [런타임] → [세션 다시 시작] 을 씁니다.

배경으로 돌리기 (daemon)

```
import threading, time

def bg():
    while True:
        print("tick")
        time.sleep(0.5)

t = threading.Thread(target=bg,
                     daemon=True)

t.start()
time.sleep(2)
print("메인 끝")
```

핵심 포인트

- ▶ daemon=True 는 '메인이 끝나면 같이 끝' 입니다.
- ▶ 이게 없으면 끝없이 도는 스레드가 남습니다.
- ▶ 몇 번 찍힐지는 매번 조금씩 다릅니다.
- ▶ 저장 같은 중요한 일은 daemon 으로 두지 않습니다.

여럿이 같은 값을 고치면

- 값을 하나 올리는 일은 사실 세 걸음입니다.
- ① 지금 값을 읽고 ② 1을 더하고 ③ 다시 씁니다.
- 두 일꾼이 ①을 동시에 하면 같은 값을 읽습니다.
- 그러면 두 번 더했는데 한 번만 오릅니다.



용어 노트

경쟁 상태(race condition) — 누가 먼저 하느냐에 따라 답이 달라지는 상태. 두 사람이 같은 칸에 동시에 적는 것과 같습니다.

값이 깨지는 것을 눈으로 보기

```
count = 0

def add():
    global count
    for _ in range(5):
        value = count      # ① 읽고
        time.sleep(0.01)   # ② 따짓하고
        count = value + 1  # ③ 쓴다

# 일꾼 둘이 5번씩 더하면
# 10 이어야 하는데
# 5 가 나옵니다.
```

핵심 포인트

- ▶ ②의 틈 동안 다른 일꾼이 같은 값을 읽습니다.
- ▶ 둘 다 0을 읽고 둘 다 1을 씁니다.
- ▶ 두 번 더했는데 1만 오른 셈입니다.
- ▶ 오류가 안 나고 답만 틀리니 찾기 어렵습니다.

자물쇠로 막기 (Lock)

```
count = 0
lock = threading.Lock()

def add():
    global count
    for _ in range(5):
        with lock:
            value = count
            time.sleep(0.01)
            count = value + 1
```

이제 항상 10 이 나옵니다.

핵심 포인트

- ▶ with lock: 안에는 한 번에 한 명만 들어갑니다.
- ▶ 먼저 들어간 일꾼이 나올 때까지 나머지는 기다립니다.
- ▶ ①②③이 끊기지 않고 한 묶음으로 끝납니다.
- ▶ 묶는 범위는 꼭 필요한 만큼만 좁게 잡습니다.

자물쇠는 좁게 겁니다

- 자물쇠 안에 들어간 동안은 다른 일꾼이 멈춰 섭니다.
- 그래서 넓게 걸면 동시에 하는 의미가 사라집니다.
- 같이 고치는 값 근처만 감쌉니다.
- 가장 좋은 것은 같이 고치는 값을 아예 줄이는 것입니다.



용어 노트

임계구역(critical section) — 한 번에 한 명만 들어가야 하는 코드 구간. 화장실 한 칸과 같습니다.

미니게임 1. 로딩 표시 만들기

해야 할 것

- ▶ 점을 찍는 일을 스레드로 돌립니다.
- ▶ 메인은 그동안 다른 일을 합니다.
- ▶ 메인의 끝나면 표시도 멈추게 합니다.
- ▶ 끝내는 조건을 반드시 넣습니다.

✓ 체크

점이 계속 찍히다가 '완료!' 가 나오며 멈춥니다.

실행 코드

```
import threading, time
running = True
def spinner():
    while running:
        print(".", end="",
              flush=True)
        time.sleep(0.2)
t = threading.Thread(
    target=spinner)
t.start()

time.sleep(2)    # 메인 작업
running = False
t.join()
print("완료!")
```

미니게임 2. 제한 시간 퀴즈

해야 할 것

- ▶ 시간을 재는 쓰레드를 배경으로 돌립니다.
- ▶ 메인은 답을 입력받습니다.
- ▶ 입력이 끝난 뒤 시간 초과였는지 확인합니다.
- ▶ 제한 시간을 바꿔 가며 해 봅니다.

✓ 체크

5초 안에 답하면 통과, 늦으면 시간 초과가 나옵니다.

실행 코드

```
import threading, time
timeout = False
def timer():
    global timeout
    time.sleep(5)
    timeout = True
threading.Thread(
    target=timer,
    daemon=True).start()
ans = input("3 + 4 = ")

if timeout:
    print("시간 초과!")
else:
    print("입력:", ans)
```

더 알아보기 (시험 범위 아님)

- Queue — 값을 직접 고치지 않고 쪽지로 주고받기.
- Event — 끝내라는 신호를 깔끔하게 보내기.
- 둘 다 오늘 배운 것을 더 안전하게 만드는 도구입니다.
- 관심 있으면 찾아보되 시험에는 나오지 않습니다.
- 오늘은 Thread · join · Lock 세 가지면 충분합니다.

With AI: 동시성은 검증이 특히 중요

- 쓰레드 코드는 돌 때마다 결과가 달라질 수 있습니다.
- 한 번 잘 돌았다고 맞는 코드가 아닙니다.
- AI에게 '어디가 위험한지' 먼저 물어봅니다.
- 받은 코드는 여러 번 실행해 확인합니다.
- 프롬프트 → 반영 → 검증을 기록합니다.



용어 노트

프롬프트(prompt) — AI에게 주는 질문이나 지시. 자세할수록 답이 좋아집니다.

프롬프트 템플릿 (위험한 곳 찾기)

- ① 제가 쓴 쓰레드 코드예요:
(코드 전체를 붙여넣기)
- ② 하려던 일:
5초 제한 퀴즈를 만들려고 해요.
- ③ 여러 쓰레드가 같이 건드리는
값이 있는지 알려 주세요.
- ④ 실행할 때마다 결과가 달라질
수 있는 곳을 짚어 주세요.
- ⑤ 끝내는 조건이 빠지지
않았는지 확인하고,
고칠 곳만 알려 주세요.

핵심 포인트

- ▶ ③ 같이 건드리는 값이 곧 위험한 곳입니다.
- ▶ ④ 는 여러 번 실행해 직접 확인해야 합니다.
- ▶ ⑤ 끝내는 조건은 코랩에서 특히 중요합니다.
- ▶ 전체 코드를 새로 받지 말고 고칠 곳만 받습니다.

실습 1. 끝나지 않는 쓰레드 고치기

해야 할 것

- ▶ 아래 코드를 실행하면 멈추지 않습니다.
- ▶ 칸 왼쪽 ■ 중지 단추를 눌러 멈춥니다.
- ▶ 무엇이 끝을 만들어 주지 않는지 찾습니다.
- ▶ 끝내는 조건을 넣어 스스로 멈추게 합니다.

✓ 체크

tick 이 몇 번 찍히고 스스로 멈춥니다.

실행 코드

```
import threading, time
def bg():
    while True:
        print("tick")
        time.sleep(0.5)
t = threading.Thread(target=bg)
t.start()
[ 무슨 일이 나나요 ]
끝없이 tick 이 나옵니다.
```

[힌트]

끝나는 조건을 쓰거나
daemon=True 를 줍니다.

※ 반드시 ■ 중지를 누르세요.

실습 2. 깨지는 값 고치기

해야 할 것

- ▶ 자물쇠 없는 코드를 실행해 5가 나오는 것을 봅니다.
- ▶ 왜 10이 아닌지 세 걸음으로 설명해 봅니다.
- ▶ with lock: 을 알맞은 자리에 넣습니다.
- ▶ 다시 실행해 10이 나오는지 확인합니다.

✓ 체크

여러 번 실행해도 항상 10 이 나옵니다.

실행 코드

```
def add():  
    global count  
    for _ in range(5):  
        value = count  
        time.sleep(0.01)  
        count = value + 1
```

[자물쇠 없이]

5

[자물쇠를 걸면]

10

[힌트]

세 줄을 with lock:
안으로 넣습니다.

실습 3. AI와 위험한 곳 점검하기

해야 할 것

- ▶ 미니게임 2의 코드를 AI에게 붙여 줍니다.
- ▶ 앞 장 템플릿으로 위험한 곳을 물어봅니다.
- ▶ 받은 답 중 하나를 직접 실험해 확인합니다.
- ▶ 확인한 것과 못 한 것을 나눠 적습니다.

✓ 체크

AI가 짚은 것 중 실제로 확인한 것을 가려냅니다.

확인해 볼 것

AI가 이렇게 말하면

직접 확인해 보세요.

"timeout 에 경쟁 상태가
있습니다"

→ 정말 그런가요?

쓰는 쪽은 하나뿐

아닌가요?

"타이머가 입력을
끊습니다"

→ 5초 뒤에 정말

입력이 끊기나요?

직접 해 보세요.

※ 그럴듯한 말과 확인된
사실은 다릅니다.

AI 답변 확인하는 법 (동시성편)

- 여러 번 실행해도 같은 결과가 나오는가?
- 끝내는 조건이 들어 있는가?
- 여러 쓰레드가 같이 고치는 값이 있는가?
- 자물쇠를 필요한 곳에만 좁게 걸었는가?
- 코드를 한 줄씩 내 말로 설명할 수 있는가?

미니 퀴즈 (5문항)

- 1) 쓰레드를 시작시키는 것은? (start / join)
- 2) 끝날 때까지 기다리는 것은?
- 3) 값 하나 올리는 일은 몇 걸음일까요?
- 4) 같이 고치는 값을 지키는 도구는?
- 5) daemon=True 는 무슨 뜻일까요?

정답: 1-start, 2-join, 3-세 걸음, 4-Lock, 5-메인이 끝나면 같이 끝

과제

- 필수 — 제한 시간 퀴즈를 완성합니다.
- 문제를 세 개 내고 맞힌 개수를 셉니다.
- 제한 시간을 넘기면 오답으로 처리합니다.
- 끝내는 조건을 반드시 넣습니다.
- AI 기록 — 프롬프트, 반영, 검증을 적습니다.

오늘 정리 & 다음 주

- 쓰레드는 기다림이 많은 일에 효과가 큽니다.
- start 로 시작하고 join 으로 기다립니다.
- 같이 고치는 값은 자물쇠로 지킵니다.
- 끝내는 조건이 없으면 코랩에서 계속 돕니다.
- 다음 주 — 내가 만든 프로그램 발표.